ASSIGNMENT 1 – CO1 | TASK 1

Understand the Problem and Identify the Objects

Scenario: Namma Metro Smart-Card Fare System

Objective / explanation

This task models the metro system using objects. SmartCard stores a balance and protects the Rs 50 minimum-balance rule. Fare calculation and card deduction are separated.

C++ source code

```
#include &lt;iostream&gt;
#include &lt;string&gt;
using namespace std;

class SmartCard {
    string cardId;
    long long balancePaise;
public:
    SmartCard(string id, long long balance) : cardId(id), balancePaise(balance) {}
    bool canTravel(long long fare) const {
        return fare &gt;= 0 &amp;&amp; balancePaise &gt;= fare &amp;&amp; balancePaise - fare &gt;= 5000;
    }
    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise -= fare;
        return true;
    }
    long long balance() const { return balancePaise; }
};

int main() {
    SmartCard card("SC101", 10000);
    long long fare = 1200;
    cout &lt;&lt; "Can travel: " &lt;&lt; (card.canTravel(fare) ? "Yes" : "No") &lt;&lt; '\n';
    if (card.deduct(fare)) cout &lt;&lt; "Balance paise: " &lt;&lt; card.balance() &lt;&lt; '\n';
}
```

Compilation

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o program
```

Actual captured program output

```
Can travel: Yes
Balance paise: 8800
```

Output screenshot

Analysis / complexity

Objects and invariants: the card accepts a trip only when the remaining balance is at least 5000 paise (Rs 50). Both the check and deduction take O(1) time.

Evidence note: Output shown is captured by executing this source in the generation environment. Recompile and run on your own system, and use your own screenshots if your instructor requires evidence from your computer.